



KRIA KR260 ROBOTICS STARTER KIT: THE FIRST STEPS

Authors: Bevilacqua Mattia, Gabriele Manduchi

Date : June 25, 2026

Contents

1	Introduction	4
1	Requirements	4
2	Environment configuration	4
2	Bare Metal Programming	6
1	Vivado	6
1.1	Project creation	7
1.2	Block configuration	8
2	JTAG interface	9
2.1	JTAG programming	9
2.2	JTAG debugging	9
3	Operative System Interface	10
1	Petalinux project	10
1.1	Hardware configuration	10
1.2	Packages configuration	11
4	Direct Memory Access - DMA - implementation	14
1	Vivado project	14
2	Petalinux project	15
2.1	Driver deployment	17

Chapter 1

Introduction

Approaching the world of Filed Programmable Gate Arrays (FPGAs) programming, specially when combined with micro processor units (uPs) to form the so-called System on Modules (SoMs), is far from being a straight procedure and one can easily get stuck trying to solve problems that can be avoided with a bit of experience. The aim of this guide is thus to share the experience and knowledge gained while working on such devices and give some standard procedure on how to approach and solve the problems that may be encountered. Some useful links are reported in the list below:

- **Petalinux Build Host packages:** https://adaptivesupport.amd.com/s/article/73296?language=en_US
- **AMD download page:** <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-design-tools/archive.html>

This guide presents an overview of four projects. Chapter 2 introduces the first project, which involves creating a classic blinking LED using only FPGA programming. Chapter 3 revisits the same project but incorporates a custom operating system (OS) configuration. While this addition is not particularly necessary for the blinking LED, it establishes a foundation for the next projects. Chapter 4 focuses on implementing two First-In, First-Out (FIFO) memories to enhance communication capabilities. Finally, chapter 5 will implement the same R/W operations using the Direct Memory Access (DMA).

1 Requirements

To program FPGA devices, software tools are usually provided by the manufacturer. Being hybrid devices, it is necessary to set up a custom environment through which the required design steps can be carried out and the final result translated into a language understandable by the device. The board used in this guide is the Xilinx Kria KR260 Robotics Starter Kit, which needs a toolchain in order to be fully exploited. Fundamental part of the toolchain are:

- Vivado (v2024.1).
- Petalinux SDK (v2022.2).
- Linux based OS (for this guide CentOS stream 9 has been used).

Newest software releases might not be compatible with the procedure(s) listed and/or with the files used to support the processes. It is recommended to check resources availability and compatibility before the installation of both Vivado and Petalinux SDK.

2 Environment configuration

The installation of Vivado is a straightforward process with no complex steps, requiring no special attention. Downloading it from the Xilinx download page should be an easy step, thus it is not reported below. On the other hand, more attention is given to the configuration of Petalinux SDK. Before the actual installation, there are few steps needed to configure the foundations:

1. Enable the 32-bit support. The 32-bit architecture is required on the host machine because some PetaLinux tools and dependencies rely on 32-bit libraries. From the terminal window type

```
1 sudo yum install glibc.i686 libstdc++.i686 ncurses-libs.i686 zlib.i686
```

In the case the host machine is running on Ubuntu, the command to install the required architecture is

```
1 sudo dpkg --add-architecture i386
2 dpkg --print-foreign-architectures
```

2. Petalinux Build Host packages. These files are required to install all the secondary packages necessary for running PetaLinux commands on the system. First, create a directory where the PetaLinux SDK will be installed. From the Petalinux Build Host packages page download the file named *plnx-env-setup.sh* and save it into the folder just created. To start the installation of all the packages type

```
1 chmod +x <installer_file_name>
2 sudo ./<file_name>
```

Once it is completed, all the packages should have been installed and the system should be ready for the Petalinux SDK installation. From the AMD download page, download the needed version and move it to the installation folder. From this page also download the .BPS file relative to the board to be used. This latter file has to be moved into the folder in which the Petalinux project are saved. By typing

```
1 chmod +x <installer_file_name>
2 sudo ./<file_name>
```

the installation should begin. By following the instruction, the installation process should proceed without any inconvenience, even though it might take a while. Finally, to set the Petalinux environment into the directory in which it is installed, type

```
1 source settings.sh
```

Note that this latter step is needed each time a new project is created.

Chapter 2

Bare Metal Programming

Bare-Metal Programming (BMP) is a common approach in embedded systems where software runs directly on hardware without requiring an operating system. This method provides direct control over hardware resources, ensuring low latency and high performance. Additionally, debugging techniques are explored to analyze and optimize the code execution.

1 Vivado

Vivado is a software developed by AMD, oriented to programming FPGAs, which supports various board and modules. The example project that will be discussed within this chapter is meant only to give an introduction to the Vivado-FPGA programming world, which is way more vast and deeper. Figure 2.1 presents a simple

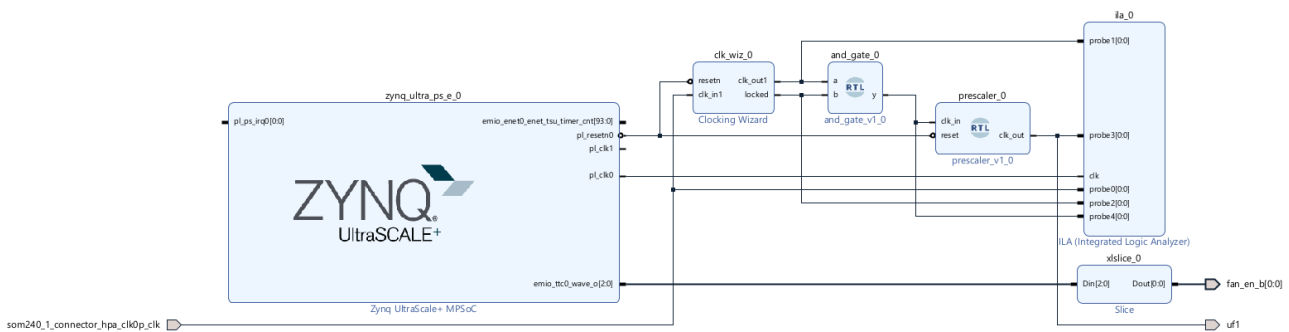


Figure 2.1: Block scheme of the Vivado design.

blinking LED design. In addition to the standard blocks imported from the example on which the design is based, the project includes several additional blocks required for proper functionality. Even though in the design is reported also the system processor block, the procedure will not deepen the Petalinux configuration, topic which will be touched in the next chapter.

Clocking Wizard The Clocking Wizard is responsible for the generation of a 10 MHz clock source within the project. The IP is imported from the IP list of Vivado, thus no code is available for consultation. However, a block configuration procedure is needed to define the block behavior.

And Gate The AND gate serves to create a stable clock source for the design by allowing the output only when the clocking wizard's output reaches a stable value. The VHDL implementation is given in the following code:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity and_gate is
5     Port (a, b : in STD_LOGIC;
6           y : out STD_LOGIC);
7 end and_gate;
8
9 architecture Behavioral of and_gate is
```

```

10 begin
11     y <= a and b;
12 end Behavioral;

```

Prescaler The prescaler is used to further divide the clock frequency of the Clocking Wizard's output. Its output is directly connected to the LED, hence, to be visible, the frequency has to be smaller than few tens of Hz.

```

1  -- Clock Divider
2  -- Every match, the output signal is toggled
3  -- The output frequency is given by: f_out = f_in / (2*max_val)
4
5  library IEEE;
6  use IEEE.STD_LOGIC_1164.ALL;
7  use IEEE.STD_LOGIC_ARITH.ALL;
8  use IEEE.STD_LOGIC_UNSIGNED.ALL;
9
10 entity prescaler is
11     Port (clk_in, reset : in STD_LOGIC;
12           clk_out : out STD_LOGIC);
13 end prescaler;
14
15 architecture Behavioral of prescaler is
16     constant max_val : integer := 500000;
17     signal counter : integer range 0 to max_val-1 := 0;
18     signal tmp : std_logic := '1';
19 begin
20     process(clk_in, reset)
21     begin
22         if reset = '0'
23         then
24             tmp <= '0';
25             counter <= 0;
26         else
27             if rising_edge(clk_in)
28             then
29                 if counter = max_val-1 -- Counts from 0 to max_val-1
30                 then
31                     counter <= 0;
32                     tmp <= not tmp;
33                 else
34                     counter <= counter + 1;
35                 end if;
36             end if;
37         end if;
38     end process;
39     clk_out <= tmp;
40 end Behavioral;

```

Integrated Logic Analyzer This device acts like a digital oscilloscope and allows the displaying of the design internal signal that are connected to the blocks probes.

1.1 Project creation

The project was created, as aforementioned, starting from a design example. To create the project, open Vivado main page, click on **Open Example Project** → **Next** → **Kria Starter Kit Design Example** → select the KR260 board → rename the project → **Default Bitstream** → create the project. From the **Sources** windows delete the already existent wrapper, while keeping the design. To add the and gate and prescaler design blocks, move to **Sources** → **Design Sources** → right-click on this latter one → **Add Sources**. Select **Add or create design sources** → **Next** → **Create file** → enter the file name → **Finish**. Once it is done, a window to configure the inputs and the outputs of the gate should open. Such procedure can be skipped, providing that those information are defined in the description.

The Physical Constraint File (PCF) defines the mapping between the FPGA's internal logic and the physical pins of the development board. It specifies which FPGA pin is connected to which external component, such as LEDs, switches, or communication interfaces. This file is crucial for real-life applications, ensuring that the synthesized design correctly interfaces with the hardware. The process of defining physical constraints follows a procedure similar to the one used for behavioral descriptions (**Sources** → **Constraints** → right-click on this latter one → **Add Sources** → **Add or create constraints** → **Next** → **Create file** → enter the file name → **Finish**), but instead of describing logic functionality, it focuses on hardware connectivity and signal routing.

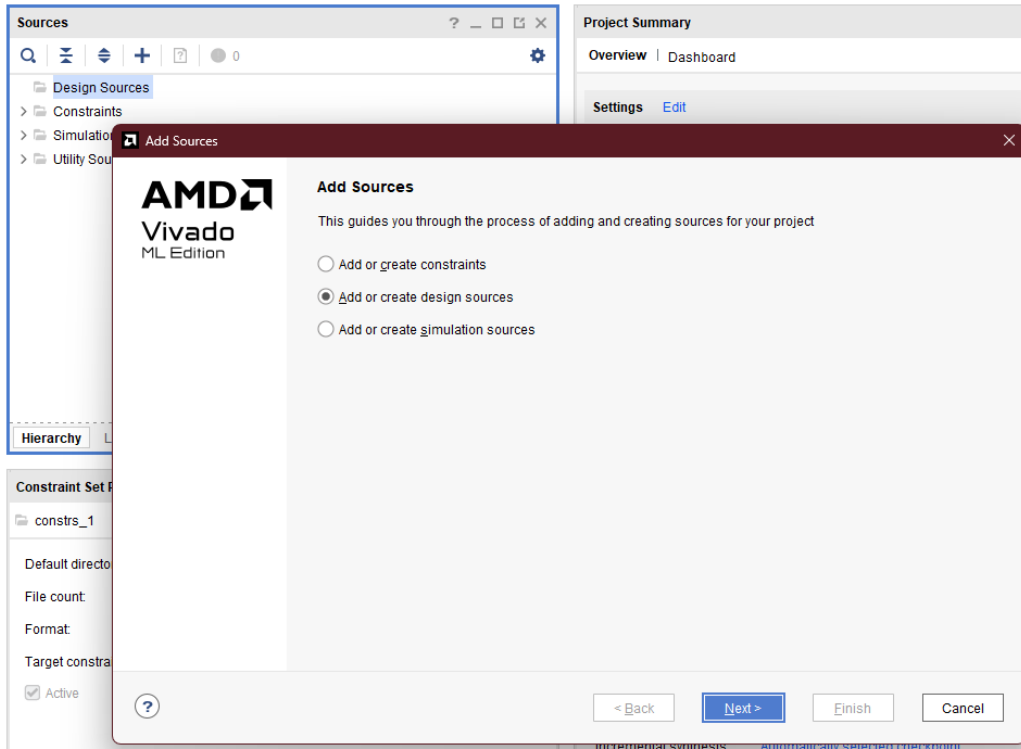


Figure 2.2: Block scheme of the Vivado design.

Proper constraint definition prevents issues like incorrect signal assignment, which could lead to malfunctioning hardware or damage to the FPGA. The code to be written is:

```
1 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
2
3 #User defined pins
4 set_property PACKAGE_PIN F8 [get_ports {uf1}]
5 set_property IOSTANDARD LVCMOS18 [get_ports {uf1}]
```

A bit of explanation is needed to understand what these few lines of code do. The first line specifies that the bitstream generated for this project will be compressed, which helps reduce the file size, making it more efficient for storage and deployment. The subsequent lines define the physical connections between the FPGA and the hardware components (such as an LED) based on the datasheet of the development board. One line specifies that the LED is connected to a specific pin on the FPGA. This pin is defined as a standard I/O pin with a voltage logic of 1.8V (LVC-MOS18), which means that the pin uses Low Voltage CMOS (LVCMOS) logic at 1.8V to communicate with the LED. These settings ensure the correct functionality of the LED based on the voltage and current requirements of the board and peripheral components.

Under the section of **IP INTEGRATOR**, click on **Open Block Design**: a new window will open. Move back to **Design Sources** → right click on the description file(s) created earlier → **Add Module to Block Design**. To create a physical connection to the board pins, move the mouse tip on the block pins and type **Ctrl+T** to add a connection. By clicking on the created pins, a window named **External Port Properties** should appear. Rename those pins with the name defined in the constraints file. Click on the + sign on the top of the design window, search and add the clocking wizard IP block. Now the design should have the same blocks displayed in Figure 2.1.

1.2 Block configuration

It should be now possible to run the automatic connection procedure (green tab on the top of the window). As previously explained, there is the necessity to configure the clocking wizard block (double click on the IP): set the reset to be active low, set the output clock to be the value that you want (there is, however, a defined range of possible clock frequencies with specific values) and tick the section in which the input and output clock are in-phase. For this example the output frequency is set to be $f_{clk-wiz} = 10MHz$. With the prescaler VHDL code defined above, the clock frequency should be equal to

$$f_{blink} = \frac{f_{clk-wiz}}{2ma_{val}} = 10Hz \quad (1.1)$$

Connect then the and gate, the prescaler and the ILA block as shown in Figure 2.1.

2 JTAG interface

The Joint Test Action Group (JTAG) interface is primarily used for testing, debugging and programming FPGA devices during the development. It is a crucial tool in the development flow when working with Xilinx FPGAs, as it allows direct communication between a computer (running Vivado) and the FPGA. Its key features are a) FPGA programming, b) Debugging, c) Boundary-scan testing and d) Accessing internal signals (ILA core).

2.1 JTAG programming

To program the FPGA, a file similar to the .hex file used for programming uPs (e.g., ATmega, STM32) is required. However, for FPGA programming, this file is usually in the form of a bitstream file (commonly with extensions like .bit or .bin). The bitstream file contains the configuration data that programs the FPGA, mapping the logical design (created in VHDL, Verilog, or through a high-level tool like Vivado) onto the FPGA's hardware resources. Unlike microcontrollers, where code is executed sequentially, an FPGA is configured with a specific hardware design that defines the logic and behavior of the entire device. Move to the **PROGRAM AND DEBUG** section and click on **Generate Bitstream**. Follow the procedure and let the bitstream to be generated. The last step is to effectively program the device. Open the **Open Hardware Manager** and click on **Open Target** → **Auto Connect**, then click on **Program Device**.

2.2 JTAG debugging

After having programmed the FPGA, opene the **Hardware Manager** and connected the device, Vivado should automatically find the debug-probes file (if not, just re-program the FPGA). Between the various signals, chose one to be referred as trigger and define the trigger event (equal to a value, rising or falling edge...) and run the trigger event for the ILA core. A window similar to that of Figure 2.3 should appear.

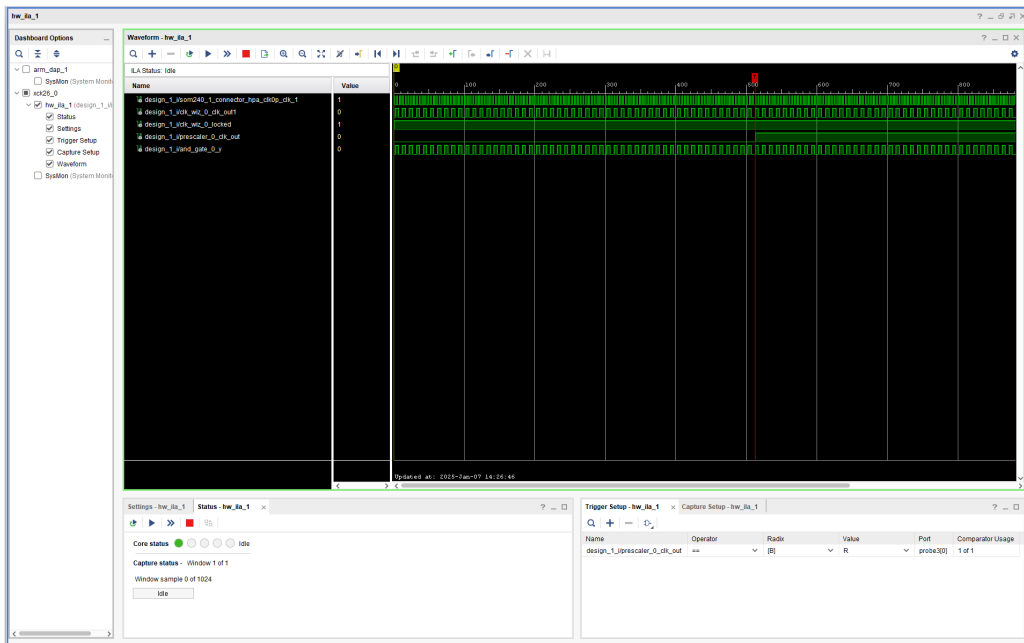


Figure 2.3: J-tag debugging window.

Chapter 3

Operative System Interface

Once the fundamental process of programming the FPGA is understood, the next step is to explore how to fully exploit the capabilities of the SoM. This chapter provides a foundational overview, but it represents only a fraction of what can be achieved.

Within this chapter it is proposed the same blinking project, but the programming of the FPGA is performed directly by the uP. This latter steps need the configuration of a custom OS through Petalinux SDK. The system is said to be custom because at the end of the procedure, the OS will have enabled only the needed packages, thus it will results in a more lighter implementation with respect to an already available OS. To program the FPGA directly from the OS, a configuration file in a binary format is needed, thus the .bit file has to be converted. There are few ways to do so, but certainly the easiest is to add to the Vivado output the creation of the .bin file. From the previous project, go to **Settings** → **Bitstream** → **bin_file** → **Apply** and regenerate the bistream. The .bin file should be saved in the same folder as the .bit file (*project_name/project_name.runs/impl_1*). In addition, also the hardware configuration file (.xsa) has to be generated (**File** → **Export** → **Export Hardware** → **Include Bitstream**).

1 Petalinux project

From within the Petalinux installation folder type

```
1 source ./settings.sh
```

to initialize the environment. To create now the project related to the exported hardware file, type

```
1 petalinux-create --type project -s <path_to_bsp_file> --name <project_name>
```

in which the .BSP file is saved in the same folder as the project and once it is done, move into the folder just created. The following two steps are quite important, so it is recommended to pay a bit more of attention.

1.1 Hardware configuration

The first configuration procedure aims to configure the core of the OS, i.e. to which architecture is addressed and which features are to be enabled. With the command

```
1 petalinux-config --get-hw-description <path_to_xsa_file>
```

a menu window will open (see Figure 3.1), from which, moving into the various directories, the following packages must be enabled (or disabled):

- **FPGA Manager** → **[*]FPGA Manager** (Enabled)
- **Image Packing Configuration** → **[]Copy final images to tftboot** (Disabled)

The FPGA manager will allow the FPGA to be programmed even during the operation of the board with few commands. However, during the creation of the image files, a warning message may appear stating that the .bit file wasn't loaded into the .wic image. To build the project type

```
1 petalinux-build
```

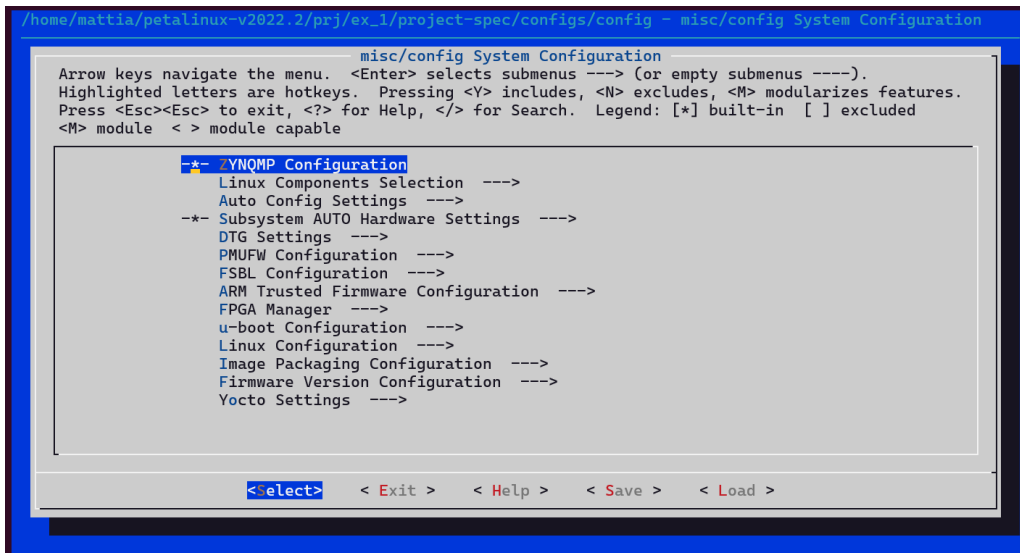


Figure 3.1: Hardware configuration window.

1.2 Packages configuration

For the projects described in this guide, the root filesystem does not need the complete XRT/OpenCL acceleration stack. The target system must provide the basic Linux utilities, SSH access, native C/C++ compilation tools and the packages required to deploy custom FPGA applications based on FPGA Manager, device-tree overlays and kernel modules. Keeping the root filesystem minimal reduces both build time and image size.

The installation menu will open with the command

```
1 petalinux-config -c rootfs
```

from which the following packages are recommended:

- **Filesystem Packages** → **base** → **util-linux** (*optional, if available*)
- **Filesystem Packages** → **console** → **utils** → **git** → [*]**git**
- **Filesystem Packages** → **console** → **utils** → **strace**
- **Filesystem Packages** → **misc** → **packagegroup-core-buildessential** → [*]**packagegroup-core-buildessential**
- **Filesystem Packages** → **misc** → **gcc-runtime** → [*]**libstdc++**
- **Filesystem Packages** → **misc** → **gdb** → [*]**gdb**
- **Filesystem Packages** → **misc** → **glibc** → [*]**glibc**, [*]**ldd**
- **PetaLinux Package Groups** → **packagegroup-petalinux** → [*]**packagegroup-petalinux**

Dropbear and OpenSSH Only one SSH server should be enabled in the root filesystem. The KR260 BSP typically enables **Dropbear**, which is the recommended choice for this guide because it provides a lightweight SSH server and integrates well with the default PetaLinux configuration.

Enabling both **Dropbear** and **OpenSSH** at the same time may cause the `do_rootfs` task to fail due to package conflicts. Therefore, use either Dropbear or OpenSSH, but never both simultaneously.

If OpenSSH is required (for example to provide a complete SFTP server), disable Dropbear and enable the following packages:

- **openssh**
- **openssh-sshd**
- **openssh-sftp-server**

If Dropbear is used, recent OpenSSH clients may attempt to use the SFTP protocol internally when executing `scp`. If the target image does not include an SFTP server, an error similar to

```
1 /usr/libexec/sftp-server: No such file or directory
```

may appear. In this case, force the legacy SCP protocol from the host machine:

```
1 scp -O file_name petalinux@board_ip:/home/petalinux/
```

This configuration provides the following features:

- basic Linux command-line functionality;
- SSH remote access to the board;
- native C/C++ compilation directly on the KR260;
- basic debugging tools such as `gdb`, `dmesg` and `strace`;
- FPGA Manager support for loading a `.bin` bitstream at run time;
- support for deploying device-tree overlays (`.dtbo`);
- support for loading custom kernel modules (`.ko`);
- support for the DMA-based user-space applications discussed in Chapter 4.

The following packages are not required for the DMA implementation presented in this guide and can be disabled unless OpenCL/XRT acceleration, graphics or multimedia applications are explicitly needed:

- `xrt`, `xrt-dev`;
- `zocl`;
- `opencl-headers`;
- `opencl-clang`, `opencl-clang-dev`;
- `libdrm`, `libdrm-tests`, `libdrm-kms`;
- `bootgen`, `bootgen-dev`;
- `dnf`;
- `gcr`, `gcr-dev`;
- `packagegroup-petalinux-gstreamer`;
- `packagegroup-petalinux-opencv`;
- `packagegroup-petalinux-v4lutils`;
- `packagegroup-petalinux-x11`.

After having installed all the needed packages, it is time to rebuild the project with the command

```
1 petalinux-build
```

In the case of building failure with error messages not related to the implementation, try to clear out the current files by typing

```
1 petalinux-config -x mrproper
```

command which is used to deep cleaning the workspace, similar to how it's used in the Linux kernel build system.

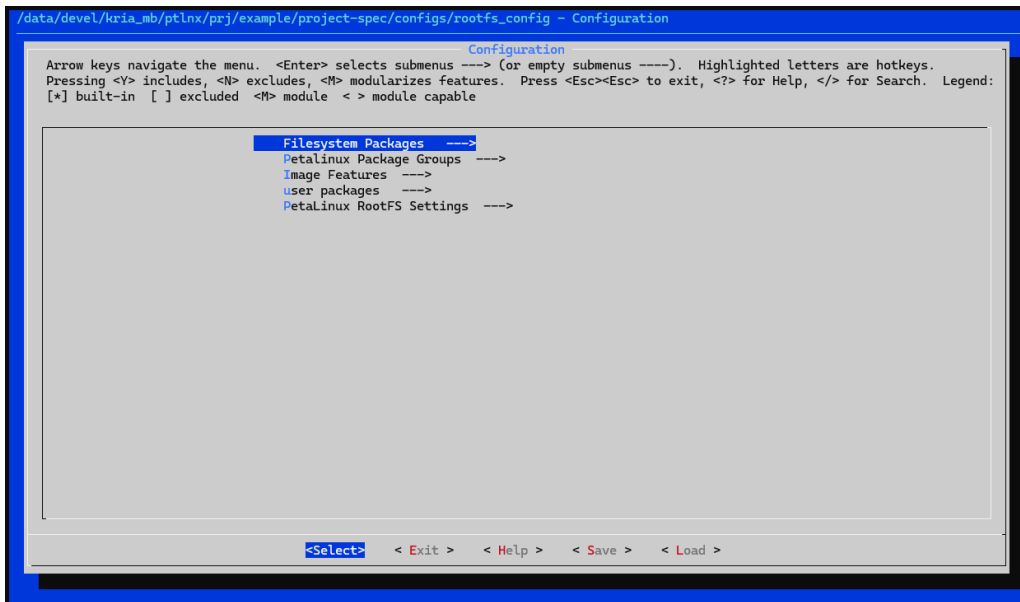


Figure 3.2: rootfs configuration window.

1.3 Board booting

Since the board boots from a SD card, the system configuration just created has to be converted into a bootable image for the card itself. First step to carry out is the creation of the image files, which are then to be flashed into the SD card. To do so, after the project build, the command is (on the same line)

```
1 petalinux-package --boot --fsbl <zynqmp_fsbl.elf> --fpga <path_to_bitstream> --u-boot
```

in which the .elf file is usually located under the path *images/linux/*. The second step is to create a .wic image for the SD card to be flashed in. The command to do that is (on the same line)

```
1 petalinux-package --wic --images-dir images/linux/ --bootfiles "ramdisk.cpio.gz.u-boot,boot.scr,Image,system.dtb,system-zynqmp-sck-kr-g-revB.dtb" --disk-name "sda"
```

from which the .wic image will be generated and found in the same folder as the .elf file. To flash the SD card make use of a third-part software, such as balenaEtcher or something similar. Once the flash is done, insert the SD in the board and power it up. With a third part software, such as PuTTY, connect to the serial port (Baud Rate 115200). The system will require to inset some credential.

1.4 SSH connection

Once the board is connected through a serial terminal and the Ethernet cable is plugged in, it is possible to find out the IP address associated to the device by typing

```
1 ip a
```

Once the IP is known, it is possible to connect to the board via SSH protocol:

```
1 ssh board_name@ip_address
```

2 FPGA programming

So far, the FPGA is not yet programmed. Besides the JTAG procedure, also the On-Run programming is of use: it allows for more flexibility and has the advantage to let the FPGA to be programmed even when the system is running, without the need to repeat all the steps previously explained. To actually program the FPGA after the system is booted, there is the need to the .bin file needs to be "manually" loaded into the FPGA through the FPGA manager. Through SSH copy the .bin file in the /lib/firmware path and to program the FPGA type

```
1 sudo sh -c 'echo "bin_file" > /sys/class/fpga_manager/fpga0/firmware'
```

The .bin file can be automatically generated by Vivado when writing the bitstream file. Go to **Settings**, then **Bitstream** and check the .bin box.

Chapter 4

Direct Memory Access - DMA - implementation

So far, the Programmable Logic (PL) and Processing System (PS) sections of the System on Module (SoM) have been utilized independently, without establishing a direct interconnection for communication or control. This project shifts the focus towards the development of a design that enables the system processor to interact with the programmable logic by reading from a specific memory address.

To achieve this, a dedicated component is implemented, which assigns a fixed memory address that supports read and write (R/W) operations. This enables seamless data exchange between the PL and PS, facilitating efficient communication. The key digital components utilized in this design include a register and a First-In, First-Out (FIFO) memory, both of which help manage data flow between the programmable logic and the processor. In addition, also the Direct Memory Access (DMA) has been exploited. The DMA is a feature in computer systems which allows hardware components to directly accessing the system memory without involving the CPU for every transfer. The data transfer is managed by a DMA controller, which is initially set by the CPU. This type of communication is used in embedded systems, high-speed networking, and storage devices to optimize performance.

1 Vivado project

As far as the creation of the Vivado project is concerned, all the steps needed to derive the final block scheme are omitted since they follow the same steps described in the previous chapters. In this section it is just shown the block design and a brief description of the blocks is given. The block scheme is composed by the following

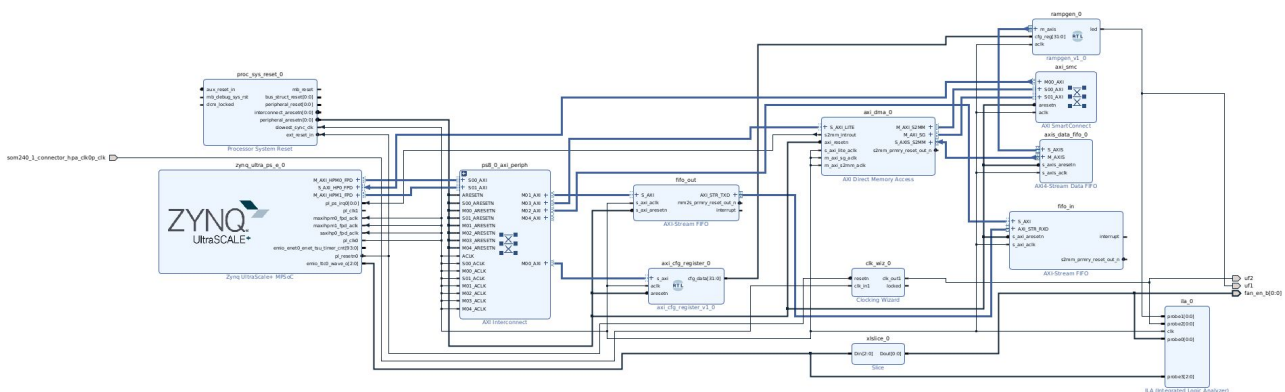


Figure 4.1: Vivado DMA block scheme.

blocks:

- **Zynq UltraScale+ Processing System:** This is a highly versatile and powerful System-on-Chip (SoC) that combines programmable logic (FPGA fabric) and processing capabilities (ARM Cortex cores). It's the brain of the system, handling computation, decision-making, and coordination between other components.
- **AXI Direct Memory Access (DMA):** This block facilitates high-speed data transfer between memory and peripherals or other components, bypassing the CPU. It's crucial for efficient data handling in high-performance systems.

- FIFO (First-In, First-Out): A type of data buffer used to ensure smooth and orderly data flow. It stores data temporarily and releases it in the same order it was received, preventing bottlenecks in the system.
- AXI Interconnect: This connects various AXI-based components within the design. It manages data flow between the processor, peripherals, memory, and other AXI-compatible blocks, ensuring efficient communication.
- Peripheral Interfaces: These are bridges connecting external devices (like sensors, displays, or communication modules) to the main system. They ensure compatibility and proper data exchange between the system and peripherals.
- Memory Controller: This manages the read and write operations to and from external or on-chip memory, ensuring data is stored and accessed efficiently.

2 Petalinux project

The root filesystem configuration required for this chapter is the minimal development configuration described in Section 1.2. In particular, XRT, ZOCL and OpenCL are not required for the custom AXI DMA flow used here. The design relies instead on FPGA Manager, a device-tree overlay, a custom kernel module and a user-space test program.

Despite having created the Vivado project and established a design flow, the current approach does not work in this case. The previous method was discovered through trial and error rather than a structured research effort like this one. The main issue arises during board boot: the DMA controller attempts to access memory addresses defined in the Vivado block design before the FPGA has been programmed. To address this, the device tree is modified after boot, creating a custom configuration to ensure proper memory access and system functionality. To upload the code relative to the DMA testing, there are few steps to follow:

- The first step is to create the file **pl-*<projectname>*.dtsi**. The file has to be modified as shown in the following code. The node *fragment@1* is copied from **pl.dtsi** (found in *components/plnx_workspace/device-tree/device-tree/*). In addition all the data relative to the DMA and the memory instantiation are copied. **dmatest_0** has to be written anew instead, since it is not found in any file.

```

1  /*
2  * CAUTION: This file is automatically generated by Xilinx.
3  * Version: XSCT
4  * Today is: Wed Feb 26 13:53:04 2025
5  */
6
7
8  /dts-v1/;
9  /plugin/;
10 / {
11     fragment@1 {
12         target = <&amba>;
13         overlay1: __overlay__ {
14             #address-cells = <2>;
15             #size-cells = <2>;
16             afi0: afi0 {
17                 compatible = "xlnx,afi-fpga";
18                 config-afi = < 0 2>, <1 2>, <2 0>, <3 0>, <4 2>, <5 2>, <6 0>, <7 0>, <8 0>,
19                 <9 0>, <10 0>, <11 0>, <12 0>, <13 0>, <14 0xa00>, <15 0x000>;
20             };
21             clocking0: clocking0 {
22                 #clock-cells = <0>;
23                 assigned-clock-rates = <999999001>;
24                 assigned-clocks = <&zynqmp_clk 71>;
25                 clock-output-names = "fabric_clk";
26                 clocks = <&zynqmp_clk 71>;
27                 compatible = "xlnx,fclk";
28             };
29             clocking1: clocking1 {
30                 #clock-cells = <0>;
31                 assigned-clock-rates = <999999001>;
32                 assigned-clocks = <&zynqmp_clk 72>;
33                 clock-output-names = "fabric_clk";
34                 clocks = <&zynqmp_clk 72>;
35                 compatible = "xlnx,fclk";
36             };
37     dmatest_0: dmatest@0 {

```



```

38     compatible = "xlnx,axi-dma-test-1.00.a";
39     dmas = <&axi_dma_0 1>;
40     dma-names = "dma0";
41 };
42 axi_cfg_register_0: axi_cfg_register@b0000000 {
43     clock-names = "aclk";
44     clocks = <&zynqmp_clk 71>;
45     compatible = "xlnx,axi-cfg-register-1.0";
46     reg = <0x0 0xb0000000 0x0 0x80>;
47 };
48 axi_dma_0: dma@a0020000 {
49     #dma-cells = <1>;
50     clock-names = "m_axi_s2mm_aclk", "m_axi_sg_aclk", "s_axi_lite_aclk";
51     clocks = <&zynqmp_clk 71>, <&zynqmp_clk 71>, <&zynqmp_clk 71>;
52     compatible = "xlnx,axi-dma-7.1", "xlnx,axi-dma-1.00.a";
53     interrupt-names = "s2mm_introut";
54     interrupt-parent = <&gic>;
55     interrupts = <0 89 4>;
56     reg = <0x0 0xa0020000 0x0 0x10000>;
57     xlnx,addrwidth = <0x20>;
58     xlnx,include-sg ;
59     xlnx,sg-length-width = <0x17>;
60     dma-channel@a0020030 {
61         compatible = "xlnx,axi-dma-s2mm-channel";
62         dma-channels = <0x1>;
63         interrupts = <0 89 4>;
64         xlnx,datawidth = <0x20>;
65         xlnx,device-id = <0x0>;
66     };
67 };
68 fifo_in: axi_fifo_mm_s@a0010000 {
69     clock-names = "s_axi_aclk";
70     clocks = <&zynqmp_clk 71>;
71     compatible = "xlnx,axi-fifo-mm-s-4.3";
72     reg = <0x0 0xa0010000 0x0 0x10000>;
73     xlnx,axi-str-rxd-protocol = "XIL_AXI_STREAM_ETH_DATA";
74     xlnx,axi-str-rxd-tdata-width = <0x20>;
75     xlnx,axi-str-txc-protocol = "XIL_AXI_STREAM_ETH_CTRL";
76     xlnx,axi-str-txc-tdata-width = <0x20>;
77     xlnx,axi-str-txd-protocol = "XIL_AXI_STREAM_ETH_DATA";
78     xlnx,axi-str-txd-tdata-width = <0x20>;
79     xlnx,axis-tdest-width = <0x4>;
80     xlnx,axis-tid-width = <0x4>;
81     xlnx,axis-tuser-width = <0x4>;
82     xlnx,data-interface-type = <0x0>;
83     xlnx,has-axis-tdest = <0x0>;
84     xlnx,has-axis-tid = <0x0>;
85     xlnx,has-axis-tkeep = <0x0>;
86     xlnx,has-axis-tstrb = <0x0>;
87     xlnx,has-axis-tuser = <0x0>;
88     xlnx,rx-cascade-height = <0x0>;
89     xlnx,rx-enable-ecc = <0x0>;
90     xlnx,rx-fifo-depth = <0x800>;
91     xlnx,rx-fifo-pe-threshold = <0x5>;
92     xlnx,rx-fifo-pf-threshold = <0x1fb>;
93     xlnx,rx-has-ecc-err-inject = <0x0>;
94     xlnx,s-axi-id-width = <0x4>;
95     xlnx,s-axi4-data-width = <0x20>;
96     xlnx,select-xpm = <0x0>;
97     xlnx,tx-cascade-height = <0x0>;
98     xlnx,tx-enable-ecc = <0x0>;
99     xlnx,tx-fifo-depth = <0x200>;
100    xlnx,tx-fifo-pe-threshold = <0x5>;
101    xlnx,tx-fifo-pf-threshold = <0x1fb>;
102    xlnx,tx-has-ecc-err-inject = <0x0>;
103    xlnx,use-rx-cut-through = <0x0>;
104    xlnx,use-rx-data = <0x1>;
105    xlnx,use-tx-ctrl = <0x0>;
106    xlnx,use-tx-cut-through = <0x0>;
107    xlnx,use-tx-data = <0x0>;
108 };
109 fifo_out: axi_fifo_mm_s@a0000000 {
110     clock-names = "s_axi_aclk";
111     clocks = <&zynqmp_clk 71>;
112     compatible = "xlnx,axi-fifo-mm-s-4.3";
113     reg = <0x0 0xa0000000 0x0 0x10000>;

```

```

114     xlnx,axi-str-rxd-protocol = "XIL_AXI_STREAM_ETH_DATA";
115     xlnx,axi-str-rxd-tdata-width = <0x20>;
116     xlnx,axi-str-txc-protocol = "XIL_AXI_STREAM_ETH_CTRL";
117     xlnx,axi-str-txc-tdata-width = <0x20>;
118     xlnx,axi-str-txd-protocol = "XIL_AXI_STREAM_ETH_DATA";
119     xlnx,axi-str-txd-tdata-width = <0x20>;
120     xlnx,axis-tdest-width = <0x4>;
121     xlnx,axis-tid-width = <0x4>;
122     xlnx,axis-tuser-width = <0x4>;
123     xlnx,data-interface-type = <0x0>;
124     xlnx,has-axis-tdest = <0x0>;
125     xlnx,has-axis-tid = <0x0>;
126     xlnx,has-axis-tkeep = <0x0>;
127     xlnx,has-axis-tstrb = <0x0>;
128     xlnx,has-axis-tuser = <0x0>;
129     xlnx,rx-cascade-height = <0x0>;
130     xlnx,rx-enable-ecc = <0x0>;
131     xlnx,rx-fifo-depth = <0x200>;
132     xlnx,rx-fifo-pe-threshold = <0x5>;
133     xlnx,rx-fifo-pf-threshold = <0x1fb>;
134     xlnx,rx-has-ecc-err-inject = <0x0>;
135     xlnx,s-axi-id-width = <0x4>;
136     xlnx,s-axi4-data-width = <0x20>;
137     xlnx,select-xpm = <0x0>;
138     xlnx,tx-cascade-height = <0x0>;
139     xlnx,tx-enable-ecc = <0x0>;
140     xlnx,tx-fifo-depth = <0x800>;
141     xlnx,tx-fifo-pe-threshold = <0x5>;
142     xlnx,tx-fifo-pf-threshold = <0x1fb>;
143     xlnx,tx-has-ecc-err-inject = <0x0>;
144     xlnx,use-rx-cut-through = <0x0>;
145     xlnx,use-rx-data = <0x0>;
146     xlnx,use-tx-ctrl = <0x0>;
147     xlnx,use-tx-cut-through = <0x1>;
148     xlnx,use-tx-data = <0x1>;
149 };
150
151 };
152 };
153 };
154

```

- The second step is to compile the file just created. Move into the folder *project-name/build/tmp/sysroots-components/x86_64/dtc-native/usr/bin/* and with the command

```

1 dtc -@ -I dts -O dtb -o pl-gabriele.dtbo pl-gabriele.dtsi
2

```

compile the new device tree. This latter command creates the .dtbo file for the device tree just created.

- The last step is to copy all the necessary files into the bard, to program the FPGA and to load the new device tree. All this steps can be executed with the following commands

```

1 sudo cp pl-gabriele.dtbo design_1_wrapper.bin /lib/firmware/
2 sh -c 'echo "design_1_wrapper.bin" > /sys/class/fpga_manager/fpga0/firmware'
3 mkdir /sys/kernel/config/device-tree/overlays/pl-gabriele
4 sh -c 'echo "pl-gabriele.dtbo" > /sys/kernel/config/device-tree/overlays/pl-gabriele/
   path'
5

```

2.1 Driver deployment

After having successfully build the new additional device tree, performing read/write operations on the memories and registers requires the definition of two additional components: a) the Driver, responsible for managing communication between the PS and the PL and b) the user program, which executes the operations using the driver to interact with the hardware. The driver must then be compiled into a .ko (Kernel Object) file, which can be loaded and executed on the KRIA board. The procedure to achieve this is quite complex, and even small mistakes can lead to errors. Therefore, careful attention is required at each step to ensure successful implementation.

An automatic Python-based generator is used to create the driver, which takes as input: a) the name of the driver b) the path to system-user.dtsi and c) the name of an input FIFO (only if read direct read operation are

needed, in the case of the DMA there it is not requested). The generator produces the driver's template files (.c and .h). The steps to correctly define deploy the driver are the following:

1. Create the driver module using the command

```
1 petalinux-create -t modules --name driver_name --enable
```

2. Copy the template files into *project-spec/meta-user/recipes-modules/driver_name/files/*.
3. Modify the Makefile to include the path to the .h file in the **MY_CFLAGS** line. Not including the path will result in an error, as PetaLinux will be unable to locate such file. The updated Makefile is shown below:

```
1 obj-m := blinkdriver.o
2
3 MY_CFLAGS += -g -DDEBUG -I /data/devel/kria_mb/ptlnx/prj/blink_mk_VIII/project-spec/
4 meta-user/recipes-modules/blinkdriver/files
5 ccflags-y += ${MY_CFLAGS}
6
7 SRC := $(shell pwd)
8
9 all:
10 $(MAKE) -C $(KERNEL_SRC) M=$(SRC)
11
12 modules_install:
13 $(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install
14
15 clean:
16 rm -f *.o *~ core .depend *.cmd *.ko *.mod.c
17 rm -f Module.markers Module.symvers modules.order
18 rm -rf .tmp_versions Modules.symvers
```

Build the driver using

```
1 petalinux-build -c driver_name
```

4. Locate the .ko file and copy it, through ssh, to the KRIA along with the .h file. Move also the .bin file.
5. Program the FPGA (see previous chapter).
6. With the command

```
1 sudo insmod driver_name.ko
```

launch the driver in the KRIA.

7. This last step has to be done after the program that will make use of the driver has been written and compiled. Once the program is done then type

```
1 ./program_name
```

and now everything should be working properly.